

Deteksi Pelanggaran Batas Kecepatan Kendaraan pada Rekaman CCTV Jalan Protokol Menggunakan YOLO11n dan ByteTrack

Aloysius Pijar Utama Indrianto, Anders Emmanuel Tan*, Evan Razzan Adyaputra*,
Kukuh Agus Hermawan*, Mahardika Ramadhana*
Department of Computer Science and Electronics, Universitas Gadjah Mada, Yogyakarta, Indonesia

Abstrak—Kecelakaan lalu lintas merupakan salah satu permasalahan keselamatan publik yang serius, dan pelanggaran batas kecepatan kendaraan menjadi salah satu faktor yang dapat meningkatkan risiko serta tingkat keparahan kecelakaan. Pengawasan kecepatan secara manual, seperti penggunaan *speed gun*, masih memiliki keterbatasan karena membutuhkan petugas di lapangan dan sulit memantau banyak kendaraan secara bersamaan. Penelitian ini mengembangkan sistem berbasis penglihatan komputer untuk mendeteksi pelanggaran batas kecepatan kendaraan pada rekaman CCTV statis. Sistem menggunakan YOLO untuk mendeteksi kendaraan pada setiap *frame*, ByteTrack untuk mempertahankan identitas kendaraan antar-*frame*, dan transformasi homografi untuk mengubah koordinat piksel ke ruang *Bird's Eye View* sehingga perpindahan kendaraan dapat dihitung dalam satuan meter. Kecepatan kendaraan dihitung berdasarkan jarak Euclidean antarposisi kendaraan pada *frame* berurutan, kemudian dihaluskan menggunakan *moving average* agar estimasi kecepatan lebih stabil. Kendaraan yang kecepatannya melebihi batas maksimum akan diklasifikasikan sebagai pelanggar, diberi indikator visual, dan dicatat bersama *timestamp*, ID kendaraan, bukti *frame*, serta nilai kecepatan. Dengan integrasi deteksi objek, pelacakan kendaraan, transformasi perspektif, estimasi kecepatan, dan pencatatan pelanggaran, sistem ini dirancang sebagai solusi otomatis yang efisien untuk membantu pemantauan lalu lintas berbasis CCTV pada konteks jalan perkotaan di Indonesia.

Kata Kunci—Pelanggaran batas kecepatan, CCTV, YOLO, ByteTrack, homografi, estimasi kecepatan, penglihatan komputer

I. PENDAHULUAN

A. Latar Belakang

Kecelakaan lalu lintas masih menjadi salah satu permasalahan keselamatan publik yang penting untuk diperhatikan. World Health Organization (WHO) melaporkan bahwa kecelakaan lalu lintas menyebabkan sekitar 1,19 juta kematian setiap tahun dan puluhan juta orang mengalami cedera non-fatal [1]. Salah satu faktor yang dapat meningkatkan risiko kecelakaan adalah pelanggaran batas kecepatan kendaraan. Kendaraan yang melaju melebihi batas kecepatan memiliki jarak pengereman yang lebih panjang, waktu reaksi pengemudi yang lebih sempit, serta potensi benturan yang lebih besar ketika kecelakaan terjadi.

Permasalahan tersebut juga relevan dengan kondisi lalu lintas di Indonesia. Berdasarkan data lalu lintas, pada tahun 2022 tercatat sekitar 131.500 kecelakaan lalu lintas di Indonesia, dengan sebagian besar kasus melibatkan kendaraan bermotor

[2]. Kondisi lalu lintas Indonesia memiliki karakteristik yang cukup kompleks, seperti kepadatan kendaraan yang tinggi, dominasi sepeda motor, variasi jenis kendaraan, serta kondisi jalan yang tidak selalu seragam. Oleh karena itu, sistem pemantauan kecepatan kendaraan perlu dirancang agar mampu bekerja pada kondisi lalu lintas yang padat dan dinamis.

Pengawasan pelanggaran kecepatan secara manual, misalnya menggunakan *speed gun*, masih memiliki beberapa keterbatasan. Metode tersebut umumnya membutuhkan petugas di lapangan, tidak selalu efektif untuk memantau banyak kendaraan secara bersamaan, dan dapat dipengaruhi oleh keterbatasan pengamatan manusia [3]. Di sisi lain, kamera CCTV telah banyak digunakan pada area jalan, persimpangan, dan titik-titik lalu lintas strategis. Rekaman CCTV tersebut dapat dimanfaatkan sebagai sumber data untuk membangun sistem deteksi pelanggaran kecepatan yang lebih otomatis, terukur, dan dapat diperiksa kembali melalui bukti visual.

Perkembangan *computer vision* memungkinkan kendaraan pada rekaman CCTV dideteksi dan dilacak secara otomatis. Dalam penelitian ini, deteksi kendaraan dilakukan menggunakan model YOLO karena model ini sesuai untuk tugas deteksi objek pada citra dan video. Secara implementasi, model dasar yang digunakan adalah YOLO11n, yaitu varian ringan dari YOLO11 yang kemudian dilatih ulang menggunakan dataset kendaraan dari rekaman CCTV. Penggunaan varian ringan dipilih agar proses deteksi tetap efisien ketika digabungkan dengan tahap pelacakan dan estimasi kecepatan.

Dataset yang digunakan pada penelitian ini merupakan dataset kustom yang diperoleh dari rekaman CCTV lalu lintas. Frame dari video CCTV diekstraksi terlebih dahulu, kemudian kendaraan pada citra diberi anotasi menggunakan Roboflow. Dataset hasil anotasi tersebut selanjutnya digunakan untuk melatih model YOLO agar lebih sesuai dengan karakteristik rekaman CCTV yang digunakan, seperti sudut pandang kamera, ukuran kendaraan pada citra, kondisi pencahayaan, dan kepadatan lalu lintas. Dengan pendekatan ini, model tidak hanya bergantung pada dataset umum, tetapi juga disesuaikan dengan kondisi data yang benar-benar digunakan dalam penelitian.

Deteksi kendaraan saja belum cukup untuk menghitung kecepatan, karena kecepatan membutuhkan informasi perubahan posisi kendaraan dari waktu ke waktu. Oleh karena itu, penelitian ini menggunakan ByteTrack untuk mempertahankan

identitas kendaraan antar-*frame*. Dengan adanya ID kendaraan yang konsisten, sistem dapat mengetahui bahwa kendaraan pada satu *frame* merupakan kendaraan yang sama dengan kendaraan pada *frame* berikutnya. Informasi ini menjadi dasar untuk menghitung perpindahan kendaraan selama interval waktu tertentu.

Tantangan lain dalam estimasi kecepatan dari rekaman CCTV adalah adanya distorsi perspektif. Jarak antarposisi kendaraan dalam satuan piksel tidak dapat langsung dianggap sebagai jarak sebenarnya di permukaan jalan. Untuk mengatasi hal tersebut, penelitian ini menggunakan transformasi homografi untuk memetakan koordinat citra kamera ke ruang *Bird's Eye View* (BEV). Setelah posisi kendaraan berada pada ruang BEV, perpindahan kendaraan dapat dikonversi ke satuan meter sehingga estimasi kecepatan menjadi lebih representatif.

Berdasarkan permasalahan tersebut, penelitian ini mengembangkan sistem deteksi pelanggaran batas kecepatan kendaraan pada rekaman CCTV menggunakan YOLO dan ByteTrack. Sistem yang dibangun terdiri dari tahap ekstraksi frame, anotasi dataset, pelatihan model deteksi kendaraan, pemilihan *Region of Interest* (ROI), deteksi kendaraan, pelacakan kendaraan, transformasi homografi, estimasi kecepatan, *smoothing*, validasi pelanggaran, dan pencatatan bukti pelanggaran. Dengan alur tersebut, sistem diharapkan dapat membantu proses pemantauan lalu lintas secara otomatis dan menyediakan informasi pelanggaran yang lebih terstruktur.

B. Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam penelitian ini adalah sebagai berikut.

- Bagaimana membangun dataset kustom kendaraan dari rekaman CCTV dan menyiapkannya untuk pelatihan model deteksi objek?
- Bagaimana menerapkan YOLO11n untuk mendeteksi kendaraan pada rekaman CCTV?
- Bagaimana menerapkan ByteTrack untuk mempertahankan identitas kendaraan antar-*frame*?
- Bagaimana menghitung kecepatan kendaraan dari rekaman CCTV dengan mempertimbangkan perubahan perspektif kamera?
- Bagaimana menentukan kendaraan yang melanggar batas kecepatan dan mencatat hasil pelanggaran agar tidak terjadi pencatatan berulang pada kendaraan yang sama?

C. Tujuan Proyek

Tujuan dari proyek ini adalah sebagai berikut.

- Membangun dataset kustom kendaraan dari rekaman CCTV melalui proses ekstraksi frame dan anotasi objek kendaraan.
- Melatih model YOLO11n agar mampu mendeteksi kendaraan pada citra CCTV yang digunakan dalam penelitian.
- Mengimplementasikan ByteTrack untuk melacak kendaraan dan mempertahankan ID kendaraan antar-*frame*.

- Menggunakan transformasi homografi untuk mengubah koordinat kendaraan dari citra kamera ke ruang *Bird's Eye View*.
- Menghitung kecepatan kendaraan berdasarkan perpindahan posisi antar-*frame* dan melakukan *smoothing* agar estimasi kecepatan lebih stabil.
- Mengembangkan mekanisme validasi dan pencatatan pelanggaran berupa ID kendaraan, waktu kejadian, bukti citra, dan nilai kecepatan.

D. Motivasi Proyek

Proyek ini dilatarbelakangi oleh kebutuhan sistem pemantauan kecepatan kendaraan yang lebih otomatis, efisien, dan sesuai dengan kondisi lalu lintas nyata. Pengawasan manual masih memiliki keterbatasan, terutama ketika harus memantau banyak kendaraan secara bersamaan. Oleh karena itu, pemanfaatan rekaman CCTV menjadi alternatif yang relevan karena kamera CCTV sudah banyak tersedia di area jalan dan persimpangan.

Melalui proyek ini, rekaman CCTV tidak hanya digunakan sebagai dokumentasi visual, tetapi juga sebagai sumber data untuk mendeteksi kendaraan, melacak pergerakannya, dan mengestimasi kecepatannya. Penggunaan dataset kustom yang dianotasi dari rekaman CCTV membuat model lebih sesuai dengan sudut pandang kamera, kondisi pencahayaan, dan karakteristik kendaraan pada lokasi pengamatan.

Dengan menggabungkan YOLO untuk deteksi kendaraan, ByteTrack untuk pelacakan antar-*frame*, serta homografi untuk konversi koordinat ke ruang *Bird's Eye View*, sistem diharapkan mampu memberikan estimasi kecepatan yang lebih terukur. Hasil deteksi pelanggaran juga dicatat dalam bentuk ID kendaraan, waktu kejadian, nilai kecepatan, dan bukti visual sehingga dapat digunakan untuk evaluasi lebih lanjut.

II. DATASET

A. Sumber Dataset

Dataset yang digunakan pada penelitian ini berasal dari rekaman CCTV lalu lintas milik Dinas Perhubungan Daerah Istimewa Yogyakarta (Dishub DIY). Rekaman diambil pada area pertigaan Bandara Adisucipto, Yogyakarta, yang merepresentasikan kondisi lalu lintas perkotaan dengan kendaraan campuran seperti sepeda motor, mobil, bus, dan truk.

Berdasarkan dataset acuan, rekaman CCTV diambil pada dua tanggal yang berbeda, yaitu 6 dan 10 Juni 2025 dengan masing-masing durasi 10 jam. Pemisahan tanggal dataset ini dilakukan untuk membedakan data *training* dan data *testing*, sehingga evaluasi kinerja sistem dapat dilakukan dengan lebih objektif. Rekaman mencakup beberapa kondisi waktu, yaitu pagi, siang, dan sore. Variasi waktu tersebut penting karena kondisi visual CCTV dapat berubah akibat perbedaan pencahayaan, kepadatan lalu lintas, oklusi antar kendaraan, getaran kamera, dan distorsi perspektif. Dengan demikian, dataset ini sesuai untuk pengembangan sistem deteksi kendaraan dan estimasi kecepatan pada kondisi lalu lintas nyata.

B. Deskripsi Dataset

Data awal berbentuk video CCTV statis. Karena model deteksi objek bekerja pada citra, video terlebih dahulu diekstraksi menjadi kumpulan *frame*. Frame hasil ekstraksi kemudian dianotasi menggunakan Roboflow dalam bentuk *bounding box*. Dataset hasil anotasi diekspor melalui Roboflow dalam format YOLOv8.

Objek yang diberi anotasi adalah kendaraan motor, mobil, bus, dan truk yang tampak pada area jalan. Objek di luar konteks lalu lintas, seperti bangunan, trotoar, pohon, dan langit, tidak dijadikan target deteksi karena tidak relevan dengan tujuan penelitian.

C. Preprocessing dan Augmentasi Dataset

Pada tahap ekspor dataset melalui Roboflow, seluruh citra diproses agar memiliki format masukan yang seragam. Preprocessing yang diterapkan meliputi *auto-orientation* untuk menyesuaikan orientasi piksel serta perubahan ukuran citra menjadi 640×640 piksel dengan metode *fit*. Ukuran ini dipilih karena sesuai dengan kebutuhan pelatihan model YOLO.

Selain preprocessing, dilakukan augmentasi untuk meningkatkan variasi data latih. Augmentasi yang digunakan meliputi penyesuaian tingkat kecerahan secara acak pada rentang -14% hingga +14% serta *Gaussian blur* acak pada rentang 0 hingga 1 piksel. Augmentasi ini bertujuan agar model lebih tahan terhadap perubahan pencahayaan dan kualitas visual pada rekaman CCTV.

D. Ukuran dan Format Dataset

Dataset hasil anotasi disusun dalam format YOLOv8. Setiap citra memiliki file label yang berisi kelas objek dan koordinat *bounding box*. Format ini digunakan karena sesuai dengan kebutuhan pelatihan model deteksi kendaraan berbasis YOLO.

Ringkasan dataset penelitian ditunjukkan pada Tabel I.

TABEL I: Ringkasan dataset penelitian

Komponen	Keterangan
Sumber data	CCTV Dishub DIY
Lokasi	Pertigaan Bandara Adisucipto, Yogyakarta
Tanggal pengambilan	6 Juni 2025 (Pelatihan) dan 10 Juni 2025 (Pengujian)
Total durasi rekaman	10 jam
Jenis data	Video CCTV dan citra hasil ekstraksi <i>frame</i>
Platform anotasi	Roboflow
Format anotasi	YOLOv8
Ukuran citra asli	1280×960 piksel
Ukuran citra setelah preprocessing dan augmentation	640×640 piksel

E. Dataset untuk Pelatihan Model

Dataset hasil anotasi digunakan untuk melatih model deteksi kendaraan. Secara spesifik, dataset yang dianotasi melalui platform Roboflow untuk tahap *training* ini bersumber dari dataset rekaman 6 Juni 2025. Pada penelitian ini, model dasar yang digunakan adalah YOLO11n. Meskipun dataset diekspor dalam format YOLOv8, format tersebut tetap dapat digunakan

untuk pelatihan model YOLO11n karena struktur labelnya mengikuti format anotasi YOLO.

YOLO11n dipilih karena memiliki ukuran yang ringan sehingga sesuai untuk pemrosesan video. Pemilihan model ringan diperlukan agar sistem tetap efisien ketika deteksi kendaraan digabungkan dengan pelacakan kendaraan menggunakan ByteTrack dan estimasi kecepatan berbasis homografi.

F. Dataset untuk Pengujian Sistem

Pada tahap pengujian, evaluasi kinerja sistem dilakukan dengan menggunakan dataset rekaman 10 Juni 2025. Rekaman CCTV diproses secara berurutan dalam bentuk *frame*. Setiap *frame* dianalisis oleh model YOLO untuk mendeteksi kendaraan. Hasil deteksi kemudian diteruskan ke ByteTrack agar kendaraan yang sama dapat diberi ID yang konsisten antar-*frame*.

Data pengujian menghasilkan beberapa keluaran, yaitu *bounding box*, kelas kendaraan, ID kendaraan, posisi kendaraan, koordinat pada ruang *Bird's Eye View* (BEV), estimasi kecepatan, dan status pelanggaran. Dengan demikian, dataset pengujian digunakan untuk mengevaluasi alur sistem secara menyeluruh, mulai dari deteksi kendaraan hingga pencatatan pelanggaran batas kecepatan.

G. Data Kalibrasi

Estimasi kecepatan dari rekaman CCTV memerlukan proses kalibrasi karena jarak dalam satuan piksel tidak dapat langsung dianggap sebagai jarak sebenarnya. Oleh karena itu, sistem menggunakan empat titik referensi pada area jalan untuk membentuk matriks homografi. Matriks ini digunakan untuk mengubah koordinat kendaraan dari citra kamera ke ruang *Bird's Eye View*.

Posisi kendaraan yang digunakan adalah titik *bottom-center* dari *bounding box*. Titik tersebut dipilih karena paling mendekati posisi kendaraan pada permukaan jalan. Setelah koordinat kendaraan berada pada ruang BEV, perpindahan kendaraan dapat dihitung dalam satuan meter dan digunakan untuk mengestimasi kecepatan.

H. Ketersediaan Dataset

Dataset yang digunakan dalam penelitian ini tersedia melalui tautan berikut:

<https://drive.google.com/drive/folders/1IubIVv7SSChvKHIX4PQ7xrLaGVNv9GmK?usp=sharing>

III. METODOLOGI

A. Input dan Preprocessing

Input pada penelitian ini berupa rekaman video dari kamera CCTV statis yang dipasang pada jalan protokol Bandara Lama, Yogyakarta. Rekaman tersebut digunakan sebagai sumber utama untuk mengamati pergerakan kendaraan pada area jalan yang diteliti. Agar dapat diproses oleh sistem, video terlebih dahulu diubah menjadi rangkaian *frame*, karena analisis pada sistem penglihatan komputer dilakukan pada citra per *frame*, bukan langsung pada aliran video secara utuh [3].

Tahap preprocessing dilakukan untuk menyiapkan data yang akan diproses oleh model deteksi serta membatasi area analisis pada bagian jalan yang relevan. Dalam penelitian ini, video diproses menggunakan laju frame asli yang kami peroleh dari metadata video yang diproses. Nilai *frames per second* tersebut digunakan untuk menghitung interval waktu antar-frame yang diperlukan pada proses estimasi kecepatan kendaraan. Selanjutnya, ditentukan *region of interest* (ROI) berupa area poligonal yang mengikuti bagian badan jalan pada citra CCTV agar sistem hanya berfokus pada area yang relevan, sedangkan bagian di luar jalan seperti trotoar, bangunan, dan langit tidak ikut dianalisis [4]. ROI ini diterapkan secara konsisten pada seluruh *frame* yang diproses. ROI digunakan sebagai batas area pengamatan. Setelah proses deteksi dilakukan, hanya kendaraan yang titik referensinya berada di dalam ROI yang diproses lebih lanjut, sedangkan deteksi di luar area tersebut diabaikan [4].

Hasil dari tahap *preprocessing* adalah *frame-frame* yang telah difokuskan pada area jalan dan siap diproses pada tahap deteksi, pelacakan, serta estimasi kecepatan kendaraan.

Secara keseluruhan, tahapan *preprocessing* meliputi ekstraksi *frame* dan penentuan ROI sebagai area pengamatan yang digunakan pada proses deteksi, pelacakan, dan estimasi kecepatan kendaraan.

B. Deteksi Kendaraan

Tiap *frame* dari video akan diekstraksi dan diberikan sebagai *input* kepada model deteksi objek YOLO. YOLO akan membagi *frame* ke dalam *grid*, tiap kotak pada *grid* merupakan *bounding box* yang memegang *confidence* yang menghitung probabilitas bahwa terdapat kendaraan bermotor pada *bouding box* tersebut [5]. Setiap *bouding box* memberikan *output* koordinat objek pada citra, *confidence*, dan probabilitas kelas (mobil, motor, truk, dan lain-lain) kepada algoritma Bytetrack, *bouding box* dengan *confidence* dibawah *threshold* tertentu tidak akan digunakan untuk mereduksi komputasi [6], [7].

Namun, beberapa *bouding box* yang berbeda kemungkinan mendeteksi kendaraan yang sama, untuk mengoptimalkan akurasi, *Non maximum suppression* (NMS) akan digunakan untuk menghapus *box* duplikat. Pada NMS, *box* dengan *confidence* tertinggi diambil sebagai deteksi yang sebenarnya, *IoU* antara *box* tersebut dan *box* lain yang mengandung duplikat akan dihitung, jika *box* memiliki *IoU* yang melebihi *threshold* tertentu maka *box* tersebut akan dihapus [8]. Untuk mengevaluasi performa prediksi YOLO, setiap *bouding box* akan menggunakan *IoU* dengan *sum-squared error* untuk menghitung *loss* antara hasil prediksi dan objek sebenarnya. Total *loss* pada YOLO dibagi menjadi 3 bagian [7]:

- *classification loss*: mengukur performa YOLO dalam menentukan probabilitas kelas (misalnya mobil=0.5, bus=0.4)
- *localization loss*: mengukur performa YOLO dalam mengidentifikasi posisi dan mengkalkulasi ukuran objek
- *confidence loss*: mengukur performa YOLO dalam menentukan jika ada objek pada *box*

C. Pelacakan Kendaraan

YOLO hanya mengidentifikasi objek dan tidak dapat menentukan apakah suatu kendaraan pada *frame* tertentu merupakan kendaraan yang sama dengan *frame* sebelumnya, Bytetrack dapat menetapkan suatu identifikasi pada tiap objek untuk menentukan jika kendaraan yang terdeteksi pada *frame* berbeda ternyata merupakan kendaraan yang sama (*tracking*). Setelah menerima input dari YOLO, Bytetrack akan membagi setiap *box* non-background ke dalam 2 *set* yang berbeda, *box* dengan *confidence* diatas *threshold* tertentu akan dikelompokkan sebagai *box* dengan *confidence* tinggi, sementara sisanya akan dikelompokkan ke dalam *box* *confidence* rendah [9]. Lalu informasi dari deteksi sebelumnya seperti posisi, kecepatan, dan ukuran dari tiap objek akan digunakan oleh Kalman Filter untuk memprediksi lokasi objek tersebut selanjutnya. Hasil prediksi tersebut akan dibandingkan dengan kelompok *box* *high confidence* terlebih dahulu, untuk metode perbandingan, *IoU* akan digunakan bersama dengan Hungarian algorithm untuk memprediksi *track*, jika nilai hasil *IoU* mencapai suatu *threshold* tertentu maka akan dianggap *match* (objek yang sama dengan *frame* sebelumnya) [9].

Algoritma bytetrack kemudian akan mencoba untuk melakukan metode perbandingan yang sama pada hasil prediksi Kalman Filter yang belum mendapatkan *match*, dengan *set* yang berisi *box* dengan *confidence* rendah. Efek *occlusion* ternyata sering mengakibatkan YOLO memberikan *confidence* rendah, pada algoritma lain deteksi dengan *confidence* rendah ini dibuang, namun bytetrack berusaha memanfaatkan informasi dengan seoptimal mungkin untuk mencapai akurasi lebih tinggi [10]. Setelah itu, *track* akan diperbarui dengan hasil *matching* yang telah diperoleh dan ID unik diberikan pada objek yang dianggap sama. Jika terdapat hasil *tracking* dari prediksi Kalman Filter yang belum mendapatkan *match* setelah dibandingkan dengan *box* *low confidence*, maka status *box* tersebut dianggap *Lost* dan kalman filter akan diaplikasikan lagi pada *track* tersebut untuk dibandingkan dengan deteksi selanjutnya. Jika *track* tersebut berada dalam *Lost* hingga durasi tertentu, maka akan dihapus [11].

D. Transformasi Ruang

Kamera CCTV menghasilkan citra dengan distorsi perspektif, sehingga pengukuran jarak dalam satuan piksel tidak dapat langsung digunakan untuk menghitung kecepatan nyata kendaraan. Untuk mengatasi masalah ini, digunakan transformasi perspektif berbasis Matriks Homografi (*H*) yang memetakan koordinat piksel pada citra kamera ke koordinat nyata dalam satuan meter pada ruang *Bird's Eye View* (BEV).

Matriks homografi *H* diperoleh dengan memilih 4 titik referensi pada area ROI yang koordinat pikselnya pada citra kamera dan koordinat nyatanya dalam meter diketahui. Dengan menggunakan fungsi `cv2.findHomography()`, matriks *H* berukuran 3×3 dihitung secara otomatis. Transformasi koordinat titik *bottom-center* kendaraan dari ruang piksel (*u, v*) ke ruang BEV (*x, y*) dalam meter dilakukan menggunakan persamaan:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \quad (1)$$

di mana koordinat BEV akhir diperoleh dengan normalisasi $x = x'/w'$ dan $y = y'/w'$. Hasil transformasi ini adalah koordinat 2D dalam satuan meter sesungguhnya yang digunakan sebagai input pada tahap kalkulasi kecepatan.

E. Kalkulasi Kecepatan dan Smoothing

Estimasi kecepatan dihitung berdasarkan kalkulasi jarak tempuh kendaraan antardua titik. Berdasarkan titik BEV yang telah dikonversi ke satuan meter, perpindahan jarak absolut (Δd) kendaraan dihitung menggunakan metrik *Euclidean Distance*:

$$\Delta d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (2)$$

Selisih waktu sebenarnya (Δt) dikalkulasi secara dinamis dengan membagi jumlah rentang *frame* yang diamati terhadap nilai FPS (*Frames Per Second*) dari metadata video. Kecepatan linear v kemudian dikonversi menjadi satuan *kilometer per hours* (km/h) menggunakan rumusan:

$$v = \left(\frac{\Delta d}{\Delta t} \right) \times 3.6 \quad (3)$$

Karakteristik *bounding box* yang dihasilkan oleh model deteksi sering mengalami fluktuasi ukuran atau (*jitter*). Menghitung kecepatan *frame* yang berurutan secara langsung akan memberikan hasil yang sangat tidak stabil. Oleh karena itu, sistem menerapkan teknik *smoothing* menggunakan *Moving Average* (jendela waktu observasi) sebanyak 6 *frame* terakhir. Penggunaan parameter 6 *frame* ini dinilai paling optimal untuk meredam *noise* getaran YOLO tanpa menimbulkan (*lag*) yang signifikan.

F. Validasi Pelanggaran dan Filter Kinematika

Tahap akhir dari *pipeline* sistem adalah memvalidasi hasil kalkulasi kecepatan untuk mencatat pelanggaran lalu lintas. Nilai kecepatan kendaraan yang telah di-*smoothing* akan dibandingkan dengan batas kecepatan (*speed limit*) sebesar 80 km/jam. Namun, untuk menekan angka (*false alarm*) yang diakibatkan oleh kesalahan model YOLO, sistem mendesain algoritma keamanan berbasis batasan fisika yang kita beri nama *Guard*.

Mekanisme filter validasi ini terbagi menjadi tiga lapis (*Guard*), yakni:

- 1) **Guard 0 (Uji Stabilitas):** Sistem mewajibkan agar kecepatan kendaraan berada dalam kondisi stabil selama 6 *frame* berturut-turut. Stabilitas didefinisikan sebagai fluktuasi deviasi antar-*frame* yang tidak lebih dari 8 km/jam.
- 2) **Guard 1 (Batas Akselerasi):** Akselerasi yang melampaui batas wajar kendaraan standar (> 4 km/jam per detik) akan diblokir, dan sistem akan mengunci data di kecepatan wajar terakhir.

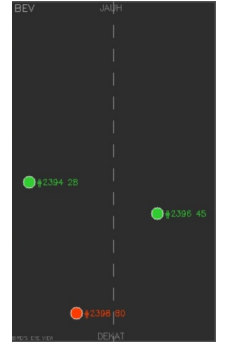
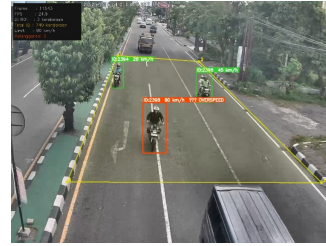
- 3) **Guard 2 (Batas Deakselerasi):** Sistem memblokir deakselerasi drastis yang tidak masuk akal (> 8 km/jam per detik) yang biasanya disebabkan oleh *track loss* parsial.

Apabila sebuah entitas ID melaju > 80 km/jam dan lolos dari ketiga *Guard* tersebut, status pelanggarannya menjadi sah. Sistem akan mengganti indikator visual kendaraan menjadi "Overspeed", menyimpan (*screenshot*), dan mencatat data (ID, waktu *timestamp*, nilai kecepatan) ke dalam direktori terminal. Untuk mencegah duplikasi pencatatan pada kendaraan pelanggar yang sama selama berada di zona ROI, sistem menerapkan mekanisme *cooldown* perekaman berdurasi 5 detik untuk setiap identitas ID kendaraan.

IV. HASIL DAN PEMBAHASAN

A. Hasil Transformasi Ruang

Secara visual, hasil transformasi ruang menggunakan matriks homografi menunjukkan bahwa distorsi sudut elevasi CCTV berhasil direduksi. Pada Gambar 1, titik-titik *Region of Interest* (ROI) yang membentuk poligon pada citra asli berhasil diproyeksikan menjadi bidang datar dua dimensi berupa *Bird's Eye View* (BEV). Proporsi kanvas BEV telah disesuaikan dengan dimensi jalan raya yang sesungguhnya (10×30 meter), sehingga perhitungan pergeseran piksel antar-*frame* dapat dikonversi ke dalam satuan meter secara proporsional.



(a) Citra Asli dengan Titik ROI (b) Proyeksi *Bird's Eye View* (BEV)

Gambar 1: Hasil pemetaan ruang perspektif ke bidang 2D.

B. Kinerja Deteksi dan Pelacakan Kendaraan

Kinerja arsitektur YOLO dan algoritma ByteTrack dievaluasi berdasarkan kemampuannya untuk mengenali dan mempertahankan identitas (ID) kendaraan yang melintas di dalam area ROI. Pengujian dilakukan dengan mengambil sampel video berdurasi 5 menit pada dua kondisi pencahayaan yang berbeda (Pagi-Siang dan Siang-Malam). Hasil perhitungan otomatis dari sistem kemudian divalidasi dengan observasi jumlah kendaraan sebenarnya yang melewati ROI, sebagaimana disajikan pada Tabel II.

TABEL II: Evaluasi Keberhasilan Pelacakan Kendaraan (Durasi 5 Menit)

Kondisi Cahaya	Total Aktual	Terdeteksi	Gagal (FN)	Akurasi (%)
Pagi-Siang	545	543	2	99.63
Siang-Malam	510	509	1	99.80
Total / Rata-rata	1055	1052	3	99.72

Berdasarkan pengujian terhadap total 1.055 populasi kendaraan, sistem mencatatkan tingkat akurasi rata-rata sebesar 99.72%. Dari ribuan entitas yang diamati, hanya terdapat 3 kasus kegagalan *tracking* (*False Negative*). Analisis visual menunjukkan bahwa kegagalan tersebut disebabkan oleh oklusi ekstrem, yakni ketika sepeda motor melaju sejajar dan tertutup sepenuhnya oleh mobil atau kendaraan besar lainnya selama melewati ROI. Tingginya akurasi membuktikan bahwa ambang batas *confidence* 0.3 yang ditetapkan pada model telah memberikan keseimbangan optimal antara sensitivitas deteksi dan pencegahan *False Positive*.

C. Akurasi Estimasi Kecepatan

Pengujian akurasi estimasi kecepatan dilakukan dengan membandingkan nilai kecepatan sistem (diambil dari nilai rata-rata tiap kendaraan di ROI) dengan perhitungan *ground truth* berbasis *manual frame-counting*. Evaluasi dilakukan menggunakan 30 sampel kendaraan random yang melintas disajikan pada Tabel III.

TABEL III: Komparasi Kecepatan Manual vs Sistem

No	ID	Jenis Kendaraan	Kondisi Cahaya	Manual (km/j)	Sistem (km/j)	Error (%)
1	4	Sepeda Motor	Pagi-Siang	51.43	52.11	1.32
2	27			57.60	57.72	0.21
3	40			57.60	60.52	5.07
4	46			53.33	53.37	0.08
5	70			57.60	57.68	0.14
6	76			51.43	53.18	3.40
7	105			49.66	50.59	1.87
8	11	Sepeda Motor	Siang-Malam	49.66	48.52	2.30
9	21			48.00	48.20	0.42
10	42			42.35	43.26	2.16
11	60			42.35	43.15	1.88
12	93			48.00	47.23	1.60
13	125			40.00	41.51	3.79
14	128			49.66	48.16	3.03
15	133			37.89	37.42	1.24
16	90	Mobil/Truck /Bus	Pagi-Siang	45.00	46.88	4.17
17	98			37.89	38.05	0.43
18	113			46.45	44.77	3.61
19	137			40.00	39.22	1.94
20	140			40.00	40.00	0.00
21	153			41.14	42.43	3.13
22	162			17.14	17.30	0.92
23	162	Mobil/Truck /Bus	Siang-Malam	43.36	42.27	2.51
24	165			42.35	41.59	1.80
25	196			43.36	44.42	2.45
26	237			45.00	45.06	0.14
27	247			45.00	45.38	0.83
28	249			45.00	44.75	0.56
29	258			38.92	38.73	0.48
30	268			22.50	24.45	8.68
Average Error (MAPE)						2.00

Berdasarkan 30 sampel yang diuji pada rentang waktu yang bervariasi, sistem mencatatkan *Mean Absolute Percentage Error* (MAPE) yang sangat rendah, yakni hanya sebesar 2.00%. Tingkat *error* yang berada jauh di bawah ambang batas toleransi operasional perangkat tilang elektronik komersial ($\pm 5\%$) ini secara nyata membuktikan bahwa proyeksi BEV dipadukan dengan *smoothing moving average* 12-frame sangat *robust* untuk kalkulasi kecepatan.

D. Validasi Pelanggaran dan Pengaruh Guard

Pada tahap pengujian awal, ditemukan adanya anomali berupa lonjakan kecepatan ekstrem (hingga melampaui 90 km/jam) yang dipicu oleh *jitter bounding box* (*jitter*) dari model YOLO. Untuk mengatasi masalah *false alarm* tersebut, sistem diimplementasikan dengan dengan *guard* dan *stability check*, yang diklasifikasikan ke dalam tiga lapis pengamanan (*Guard 0, 1, dan 2*).

Algoritma akan mengevaluasi selisih kecepatan tiap detiknya. Jika akselerasi terdeteksi melampaui batas wajar (> 4 km/jam per detik) atau deakselerasi wajar (> 8 km/jam per detik), sistem akan menolak kenaikan tersebut dan mempertahankan kecepatan di angka stabil terakhir.

Selain itu, sistem mewajibkan agar kecepatan berada dalam kondisi stabil selama 6 *frame* berturut-turut (dengan toleransi fluktuasi maksimal 8 km/jam) sebelum memvalidasi sebuah pelanggaran. Untuk membuktikan efektivitas kombinasi *guard* ini, dilakukan pengujian komparatif sebagaimana disajikan pada Tabel IV.

TABEL IV: Perbandingan Kinerja Kombinasi Guard terhadap Deteksi Pelanggaran

No	Kombinasi Guard	Total Deteksi	Pelanggaran Palsu (False Positive)	Pelanggaran Valid (True Positive)
1	Tanpa Guard	14	12	2
2	Guard 1 + 2	7	5	2
3	Guard 0 + 1 + 2	2	0	2

Berdasarkan Tabel IV, sistem tanpa perlindungan (*Tanpa Guard*) menghasilkan 14 deteksi pelanggaran, di mana mayoritas (12 kasus) merupakan *false positive* akibat *jitter*. Penggunaan *Guard 1 + 2* mampu mengurangi kesalahan tersebut, namun masih meloloskan 5 kasus deteksi palsu. Implementasi penuh ketiga lapis filter (*Guard 0 + 1 + 2*) terbukti memberikan hasil paling optimal dengan mengeliminasi 100% kasus *jitter*. Lebih lanjut, kolom *True Positive* menunjukkan angka yang konstan (2 kasus), membuktikan bahwa kombinasi filter ini bekerja sangat baik dalam meredam error sistem tanpa mengurangi sensitivitas terhadap kendaraan yang benar-benar melakukan pelanggaran batas kecepatan.

V. KESIMPULAN

Penelitian ini telah berhasil mengembangkan dan mengimplementasikan sistem deteksi pelanggaran batas kecepatan kendaraan berbasis *computer vision* tanpa bergantung pada sensor radar fisik. Penggunaan model YOLO11n yang telah melalui tahap *fine-tuning*, dipadukan dengan algoritma ByteTrack, terbukti sangat *robust* untuk mendeteksi dan melacak identitas kendaraan di jalan raya. Sistem ini mencatatkan tingkat akurasi pelacakan yang sangat tinggi, yaitu 99,72%, dengan tingkat kegagalan deteksi yang sangat minim (hanya 3 kasus dari total 1.055 populasi kendaraan).

Transformasi ruang perspektif dari kamera pengawas (CCTV) menjadi *Bird's Eye View* (BEV) menggunakan perhitungan matriks homografi terbukti sangat efektif untuk mengestimasi kecepatan secara visual. Hasil pengujian menunjukkan nilai *Mean Absolute Percentage Error* (MAPE) sebesar

2,00% yang berada jauh di bawah standar toleransi kesalahan operasional ($\pm 5\%$).

Lebih lanjut, penerapan algoritma *guard* tiga lapis (*Guard 0, 1, dan 2*) berhasil menciptakan mekanisme validasi pelanggaran yang akurat. Sistem ini secara efektif mengeliminasi 100% kasus pelanggaran palsu (*false positive*) yang diakibatkan oleh *bounding box jitter*, sehingga memastikan hanya pelanggaran valid yang dicatat oleh sistem.

DAFTAR PUSTAKA

- [1] World Health Organization, "Global status report on road safety 2023," tech. rep., World Health Organization, Geneva, Switzerland, 2023. Available: <https://www.who.int/publications/i/item/9789240086517>.
- [2] N. Kurohman and G. Student, "Indonesia's road safety crisis: Issues and policy responses," n.d. Reference cited in project proposal.
- [3] D. Fernández Llorca, A. Hernández Martínez, and I. García Daza, "Vision-based vehicle speed estimation: A survey," *IET Intelligent Transport Systems*, vol. 15, no. 8, pp. 987–1005, 2021.
- [4] M. Vasu, N. Abreu, R. Vásquez, and C. López, "Vehicle-counting with automatic region-of-interest and driving-trajectory detection," 2021. Available: <https://arxiv.org/abs/2108.07135>.
- [5] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," 2020. Available: <https://arxiv.org/abs/2004.10934>.
- [6] P. Kuntekar, Y. Narule, R. Mahajan, S. Mandlapure, E. Mehendale, and Y. Meshram, "Traffic management system using yolo algorithm," 2023. Available: https://www.researchgate.net/publication/377677383_Traffic_Management_System_Using_YOLO_Algorithm.
- [7] A. Lakshmi Rishika, C. Aishwarya, A. Sahithi, and M. Premchender, "Real-time vehicle detection and tracking using yolo-based deep sort model: A computer vision application for traffic surveillance," *Turkish Journal of Computer and Mathematics Education*, 2021. Available: <https://turcomat.org/index.php/turkbilmat/article/view/13530/9695>.
- [8] GeeksforGeeks, "What is non-maximum suppression?," 2023. Available: <https://www.geeksforgeeks.org/computer-vision/what-is-non-maximum-suppression/>.
- [9] Datature, "Introduction to bytetrack: Multi-object tracking by associating every detection box," n.d. Available: <https://datature.io/blog/introduction-to-bytetrack-multi-object-tracking-by-associating-every-detection-box>.
- [10] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "Bytetrack: Multi-object tracking by associating every detection box," 2021. Available: <https://arxiv.org/abs/2110.06864>.
- [11] Y. Zhang, X. Wang, X. Ye, W. Zhang, J. Lu, X. Tan, E. Ding, P. Sun, and J. Wang, "Bytetrackv2: 2d and 3d multi-object tracking by associating every detection box," 2023. Available: <https://arxiv.org/abs/2303.15334>.

LAMPIRAN

Repositori lengkap proyek dapat diakses melalui link berikut:

https://github.com/Plumz17/CVL_FinalProject